COMS W4771: Machine Learning

Lecture 21: Training Neural Networks

Tony Dear, Ph.D.

Department of Computer Science

School of Engineering and Applied Sciences

Topics

- Scalar, vector, and matrix gradients
- Backpropagation algorithm
- Initialization and optimizers

Problem Statement

- The output of a neural network is a composition of hidden layer outputs:

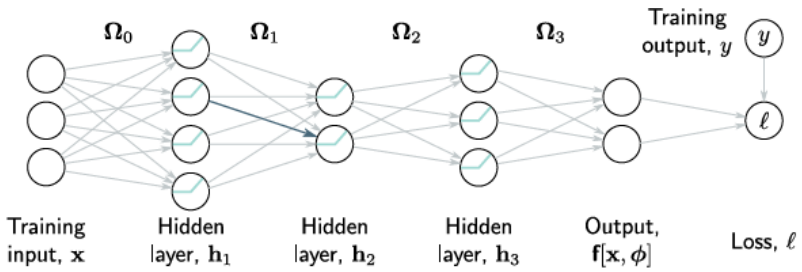
$$\mathbf{f}(\mathbf{x};\boldsymbol{\phi}) = \mathbf{a}(\boldsymbol{\beta}_K + \boldsymbol{\Omega}_K(\mathbf{a}(\boldsymbol{\beta}_{K-1} + \boldsymbol{\Omega}_{\mathrm{K}-1}(\cdots \mathbf{a}(\boldsymbol{\beta}_0 + \boldsymbol{\Omega}_0\mathbf{x})\cdots)$$

- Given a data set, we can perform prediction on each sample, compute the loss $L\big(\mathbf{y}_i, \mathbf{f}(\mathbf{x}_i;\boldsymbol{\phi})\big)$, and then use the gradients $\frac{\partial L}{\partial \boldsymbol{\phi}}$ to update parameters

- Since out networks are feed-forward, the effect of any weight on L depends directly on the hidden layer outputs that they multiply
- We should store all activations during the *forward pass* so that we can use them to compute gradients

Backward Pass

- A *change* in a weight can cause changes in the activations in subsequent layers
- So $\frac{\partial L}{\partial \mathbf{\Omega}_k}$ depends on the effects of all activations from $k+1$ to K, but has no dependence on the components that came before it
- For efficient computation of gradients, we should start with those closest to the output and work our way *backward*
- To compute $\frac{\partial L}{\partial \mathbf{\Omega}_k}$, we will need all the "later" derivatives already computed



Prince

Scalar Example

- Suppose we have a network with parameters $\boldsymbol{\phi} = (\beta_0, w_0, \dots, \beta_3, w_3)$

$$f(x; \boldsymbol{\phi}) = \beta_3 + w_3\, a_3(\beta_2 + w_2\, a_2(\beta_1 + w_1\, a_1(\beta_0 + w_0 x)))$$

- Forward pass: Compute and store all intermediate quantities $f_0 = \beta_0 + w_0 x_i$, $h_k = a_k(f_{k-1})$ and $f_k = \beta_k + w_k h_k$ for $k = 1,2,3$, and the loss l_i

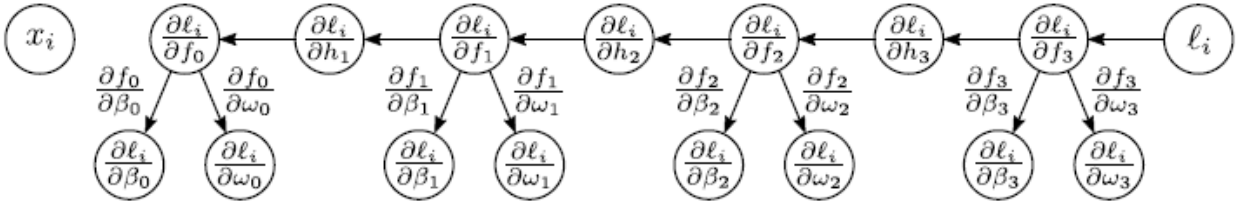
- Start by computing $\frac{\partial l_i}{\partial f_3}$; e.g., $l_i = \frac{1}{2}(y_i - f_3)^2$ has gradient $\frac{\partial l_i}{\partial f_3} = f_3 - y_i$
- From the chain rule, we can obtain $\frac{\partial l_i}{\partial h_3} = \frac{\partial f_3}{\partial h_3}\frac{\partial l_i}{\partial f_3} = w_3(f_3 - y_i)$, allowing us to reuse the gradient from the output

Scalar Example

- As we move *backward* from K to 0, we can compute $\frac{\partial l_i}{\partial h_k} = \frac{\partial f_k}{\partial h_k}\frac{\partial l_i}{\partial f_k} = w_k \frac{\partial l_i}{\partial f_k}$ and $\frac{\partial l_i}{\partial f_{k-1}} = \frac{\partial h_k}{\partial f_{k-1}}\frac{\partial l_i}{\partial h_k}$, where $\frac{\partial h_k}{\partial f_{k-1}}$ depends on the activation function a_k
- If $h_k = ReLU(f_{k-1})$, then $\frac{\partial h_k}{\partial f_{k-1}} = 1$ if $f_{k-1} > 0$ and $\frac{\partial h_k}{\partial f_{k-1}} = 0$ otherwise
- Gradients wrt network parameters can then be computed as

$$\frac{\partial l_i}{\partial \beta_k} = \frac{\partial f_k}{\partial \beta_k}\frac{\partial l_i}{\partial f_k} = \frac{\partial l_i}{\partial f_k}$$

$$\frac{\partial l_i}{\partial w_k} = \frac{\partial f_k}{\partial w_k}\frac{\partial l_i}{\partial f_k} = h_k \frac{\partial l_i}{\partial f_k}$$



Prince

Matrix Formulation

- With more than one node in a hidden layer, we have the matrix/vector relationships $\mathbf{f}_0 = \boldsymbol{\beta}_0 + \boldsymbol{\Omega}_0 \mathbf{x}_i$, $\mathbf{h}_k = \mathbf{a}_k(\mathbf{f}_{k-1})$, and $\mathbf{f}_k = \boldsymbol{\beta}_k + \boldsymbol{\Omega}_k \mathbf{h}_k$
- Gradient dimensions: $\frac{\partial l_i}{\partial \mathbf{f}_{k-1}} \in \mathbb{R}^{D_k}$, $\frac{\partial l_i}{\partial \mathbf{h}_k} \in \mathbb{R}^{D_k}$, and $\frac{\partial l_i}{\partial \mathbf{f}_k} \in \mathbb{R}^{D_{k+1}}$
- So $\frac{\partial l_i}{\partial \mathbf{h}_k} = \frac{\partial \mathbf{f}_k}{\partial \mathbf{h}_k} \frac{\partial l_i}{\partial \mathbf{f}_k}$, where $\frac{\partial \mathbf{f}_k}{\partial \mathbf{h}_k} = \boldsymbol{\Omega}_k^\top \in \mathbb{R}^{D_k \times D_{k+1}}$ since $\boldsymbol{\Omega}_k$ maps $\mathbf{h}_k$ to $\mathbf{f}_k$
- $\frac{\partial l_i}{\partial \mathbf{f}_{k-1}} = \frac{\partial \mathbf{h}_k}{\partial \mathbf{f}_{k-1}} \frac{\partial l_i}{\partial \mathbf{h}_k}$, where $\frac{\partial \mathbf{h}_k}{\partial \mathbf{f}_{k-1}} \in \mathbb{R}^{D_k \times D_k}$ is diagonal since $(\mathbf{h}_k)_i$ only depends on $(\mathbf{f}_{k-1})_i$; if $\mathbf{a}_k$ is ReLU, $\left(\frac{\partial \mathbf{h}_k}{\partial \mathbf{f}_{k-1}}\right)_{ii} = 1$ if $(\mathbf{f}_{k-1})_i > 0$

Backpropagation Algorithm

- Forward pass: Compute $\mathbf{f}_0 = \boldsymbol{\beta}_0 + \boldsymbol{\Omega}_0 \mathbf{x}_i$, $\mathbf{h}_k = \mathbf{a}_k(\mathbf{f}_{k-1})$, $\mathbf{f}_k = \boldsymbol{\beta}_k + \boldsymbol{\Omega}_k \mathbf{h}_k$
- Backward pass: Compute gradients of loss wrt $\mathbf{h}_k$ and $\mathbf{f}_k$:

$$\frac{\partial l_i}{\partial \mathbf{h}_k} = \boldsymbol{\Omega}_k^\top \frac{\partial l_i}{\partial \mathbf{f}_k} \qquad \frac{\partial l_i}{\partial \mathbf{f}_{k-1}} = \frac{\partial \mathbf{h}_k}{\partial \mathbf{f}_{k-1}} \frac{\partial l_i}{\partial \mathbf{h}_k}$$

- Compute gradients of loss wrt parameters $\boldsymbol{\beta}_k$ and $\boldsymbol{\Omega}_k$:

$$\frac{\partial l_i}{\partial \boldsymbol{\beta}_k} = \frac{\partial l_i}{\partial \mathbf{f}_k} \qquad \frac{\partial l_i}{\partial \boldsymbol{\Omega}_k} = \frac{\partial l_i}{\partial \mathbf{f}_k} \frac{\partial \mathbf{f}_k}{\partial \boldsymbol{\Omega}_k} = \frac{\partial l_i}{\partial \mathbf{f}_k} \mathbf{h}_k^\top$$

- Since we want $\frac{\partial l_i}{\partial \boldsymbol{\Omega}_k} \in \mathbb{R}^{D_{k+1} \times D_k}$, we take the *outer product* of $\frac{\partial l_i}{\partial \mathbf{f}_k} \in \mathbb{R}^{D_{k+1}}$ with $\frac{\partial \mathbf{f}_k}{\partial \boldsymbol{\Omega}_k} = \mathbf{h}_k \in \mathbb{R}^{D_k}$ to obtain the desired matrix

Sensitivity to Weights

- Due to composition, the network can be sensitive to initial weights
- Suppose we independently initialize all $\beta_i = 0$ and $\Omega_{ij} \sim \mathcal{N}\left(0, \sigma_\Omega^2\right)$

- Consider the response $(\mathbf{f}_k)_i = (\boldsymbol{\beta}_k)_i + (\boldsymbol{\Omega}_k)_{i.}\mathbf{h}_k$ at a hidden layer k:

$$E\left[(\boldsymbol{\beta}_k)_i + \sum_{j=1}^{D_k} (\boldsymbol{\Omega}_k)_{ij}\,(\mathbf{h}_k)_j\right] = E[(\boldsymbol{\beta}_k)_i] + \sum_{j=1}^{D_k} E\left[(\boldsymbol{\Omega}_k)_{ij}\right]E\left[(\mathbf{h}_k)_j\right] = \sum_{j=1}^{D_k} 0 \cdot E\left[(\mathbf{h}_k)_j\right] = 0$$

$$Var[(\mathbf{f}_k)_i] = E\left[(\mathbf{f}_k)_i^2\right] - E[(\mathbf{f}_k)_i]^2 = E\left[\left(\sum_{j=1}^{D_k} (\boldsymbol{\Omega}_k)_{ij}\,(\mathbf{h}_k)_j\right)^2\right] = \sum_{j=1}^{D_k} E\left[(\boldsymbol{\Omega}_k)_{ij}^2\right]E\left[(\mathbf{h}_k)_j^2\right]$$

- Where in both derivations we exploit independence between $\boldsymbol{\Omega}_k$ and $\mathbf{h}_k$

Change in Variance

$$Var[(\mathbf{f}_k)_i] = \sum_{j=1}^{D_k} E\left[(\mathbf{\Omega}_k)_{ij}^2\right] E\left[(\mathbf{h}_k)_j^2\right] = \sigma_\Omega^2 \sum_{j=1}^{D_k} E\left[(\mathbf{h}_k)_j^2\right]$$

- Suppose $\mathbf{h}_k = ReLU(\mathbf{f}_{k-1})$, where $\mathbf{f}_{k-1}$ is distributed symmetrically about 0
- $\mathbf{h}_k$ has equal contributions from the positive and negative halves of $(\mathbf{f}_{k-1})_j$, but $(\mathbf{h}_k)_j$ only keeps the positive half of $(\mathbf{f}_{k-1})_j$

- So $E\left[(\mathbf{h}_k)_j^2\right] = \frac{1}{2} E\left[(\mathbf{f}_{k-1})_j^2\right] = \frac{1}{2} Var\left[(\mathbf{f}_{k-1})_j\right] \triangleq \frac{1}{2} v_{k-1}$, which yields

$$Var[(\mathbf{f}_k)_i] = \sigma_\Omega^2 \sum_{j=1}^{D_k} \frac{1}{2} Var\left[(\mathbf{f}_{k-1})_j\right] = \frac{1}{2} D_k \sigma_\Omega^2 v_{k-1}$$

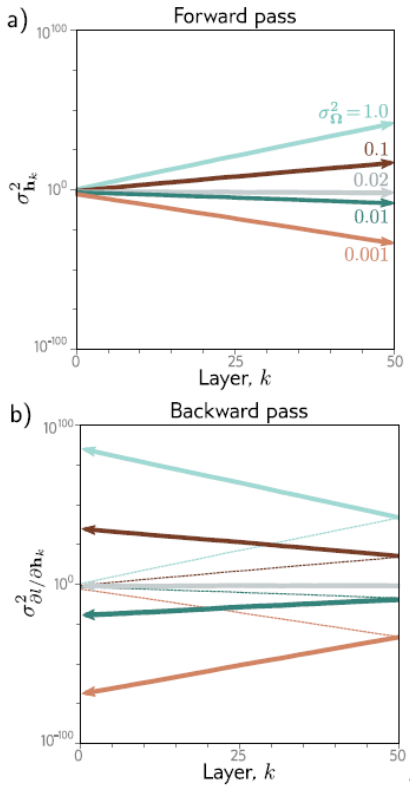
He Initialization

$$Var[(\mathbf{f}_k)_i] = \frac{1}{2} D_k \sigma_\Omega^2 v_{k-1}$$

- So $Var[(\mathbf{f}_k)_i]$ *cumulatively* scales in a deep network by the hidden layer sizes D_k and initial weight variances σ_Ω^2

- To keep these stable, **He initialization** sets $\sigma_\Omega^2 = 2/D_k$
- Or if $\Omega_k \in \mathbb{R}^{D_{k+1} \times D_k}$, set $\sigma_\Omega^2 = 4/(D_k + D_{k+1})$

- If σ_Ω^2 is much smaller or much larger, we can see *vanishing* or *exploding gradients* in the backward pass



Prince

Stochastic Gradient Descent

- We typically aim to minimize the loss over all training data, so we can compute $\frac{\partial L}{\partial \boldsymbol{\phi}} = \sum_{i=1}^{N} \frac{\partial l_i}{\partial \boldsymbol{\phi}}$ for gradient descent updates

- In practice, perform *stochastic gradient descent* with minibatches of data, so that we can make multiple parameter updates per epoch
- Each sample within a batch can be processed in parallel—very efficient!

- Since loss function is generally very non-convex, SGD can be slow
- May need to carefully tune learning rates and/or incorporate momentum

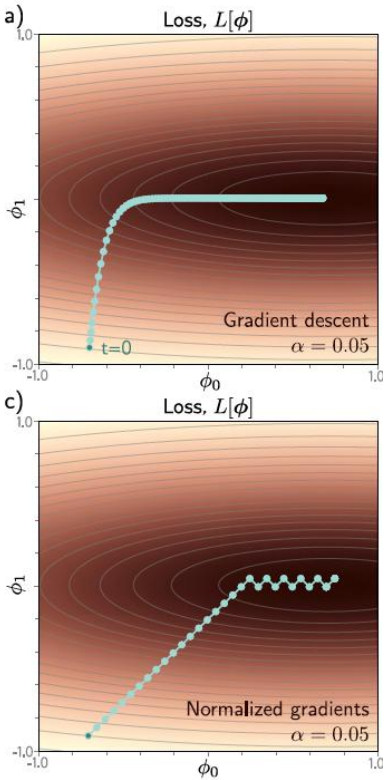
Gradient Normalization

- Even with adaptive learning rates, SGD can still be slow or unstable if parameters have different gradient scales

- We can *normalize* the gradients $\mathbf{m}_{t+1}$ so that we move a fixed distance along the computed gradient direction:

$$\boldsymbol{\phi}_{t+1} \leftarrow \boldsymbol{\phi}_t - \alpha \frac{\mathbf{m}_{t+1}}{\left(\mathbf{m}_{t+1}^2\right)^{1/2} + \epsilon}$$

- Where ϵ is a small constant to prevent division by 0
- Drawback: This will generally bounce around the minimum instead of reaching it exactly



Prince

Adam Optimizer

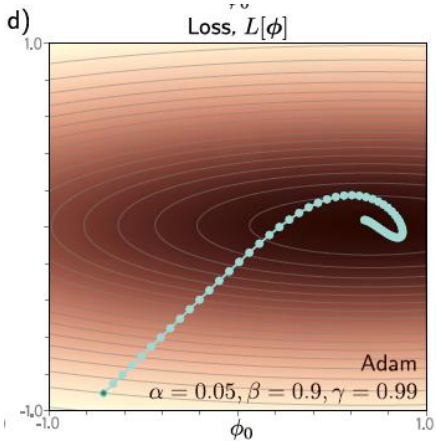
- **Adaptive moment estimator (Adam)** keeps track of momentum for both the gradient and squared gradient terms to enable "accumulation" over time:

$$\mathbf{m}_{t+1} \leftarrow \beta \mathbf{m}_t + (1-\beta)\frac{\partial L}{\partial \boldsymbol{\phi}} \qquad \mathbf{v}_{t+1} \leftarrow \gamma \mathbf{v}_t + (1-\gamma)\left(\frac{\partial L}{\partial \boldsymbol{\phi}}\right)^2$$

- Since the momenta $\mathbf{m}_t$ and $\mathbf{v}_t$ start small, we can give them an initial "boost" that fades over time:

$$\hat{\mathbf{m}}_{t+1} \leftarrow \frac{\mathbf{m}_{t+1}}{1-\beta^{t+1}} \qquad \hat{\mathbf{v}}_{t+1} \leftarrow \frac{\mathbf{v}_{t+1}}{1-\gamma^{t+1}}$$

- The overall update is then $\boldsymbol{\phi}_{t+1} \leftarrow \boldsymbol{\phi}_t - \alpha \frac{\hat{\mathbf{m}}_{t+1}}{\sqrt{\hat{\mathbf{v}}_{t+1}} + \epsilon}$



Prince

Summary

- Samples can be fed forward through a network to perform prediction
- Gradients of loss can be computed efficiently by propagating backward

- Need to initialize weights properly or normalize to ensure that values and gradients do not explode or vanish

- Can update parameters using SGD over minibatches of data
- Adam optimizer incorporates momentum and standardizes magnitudes of updates over multiple parameters